

Question 1

Incorrect

Mark 0.00 out of 1.00

[Flag question](#)

În urma execuției codului de mai jos, ia naștere o ierarhie de procese, reprezentată ca un arbore în care procesul inițial este rădăcina.

```
if(fork() != 0) {  
    fork();  
    fork();  
    fork();  
}
```

Numărul de procese din arbore este **×** [9] .

Adâncimea arborelui (ca număr de noduri de la rădăcină până la cea mai îndepărtată frunză) este

× [4] .

Numărul de frunze din arbore este **×** [5] .

Numărul maxim de copii ai unui nod este **×** [4] .

Question 2

Partially correct

Mark 0.75 out of 1.00

[Flag question](#)

Se rulează codul de mai jos:

```
unsigned char c = 0;  
int i = 0, pos = 0;  
unsigned int x = 0;  
int fd = open("test.dat", O_RDONLY);  
while(read(fd, &c, 1) == 1) {  
    printf("%02x ", c);  
    if(++i % 16 == 0) {  
        printf("\n");  
    }  
}  
printf("\n");  
pos = lseek(fd, 0, SEEK_CUR);  
printf("pos = %d\n", pos);  
lseek(fd, -4, SEEK_CUR);  
read(fd, &x, 4);  
printf("x = %d\n", x);  
lseek(fd, 0, SEEK_SET);  
read(fd, &c, 1);  
printf("c1 = %02x\n", c);  
while(read(fd, &c, 1) == 1) {  
    if(c == 0) break;  
}  
read(fd, &c, 1);  
printf("c2 = %02x\n", c);
```

Completați rezultatul afișat pe ecran.

42 47 51 52 4e 53 4b 46 56 46 43 50 48 42 44 47
57 51 45 45 00 7c 5a 42 59 45 42 57 59 48 49 43
45 55 49 53 5a 52 48 4d 51 46 45 53 56 50 49 59
4b 45 4c 59 5a 4c 52 41 20 01 00 00

pos = ✓



?

Completați rezultatul afișat pe ecran.

42 47 51 52 4e 53 4b 46 56 46 43 50 48 42 44 47
57 51 45 45 00 7c 5a 42 59 45 42 57 59 48 49 43
45 55 49 53 5a 52 48 4d 51 46 45 53 56 50 49 59
4b 45 4c 59 5a 4c 52 41 20 01 00 00



pos = ✓
x = ✗ [288]
c1 = ✓
c2 = ✓

Your answer is partially correct.

Ne aflăm în directorul curent **/home/student**. Pentru fiecare din căile de mai jos, completați varianta normalizată a acesteia, adică scrieți calea absolută spre fișierul indicat, în cel mai simplu mod posibil.

- `/media/./source/question/code/lab/../../../../laborator/lab/example/./repos/lab/../../../../problem.txt`
 ✗
[\[/media/source/laborator/lab/problem.txt\]](#)
- `sim/pages/../../../../so/pages/sim/../../../../code/Homework/./problems/Downloads/../../../../ex1.txt`
 ✗
[\[/home/code/Homework/ex1.txt\]](#)
- `question/Homework/pages/././exercise/laborator/./code.c`
 ✓
- `os/./laborator/Homework/pages/../../../../code/./os/pages/../../../../repos/code/exam/pages/older/Homework,`
 ✗
[\[/home/student/os/repos/code/exam/mycode.cpp\]](#)

Se dă un planificator de tipul **First Come, First Served**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	3	6
2	5	4
3	2	3
4	0	2

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 1.5 ✖ [2] .

0	4	✓
1	4	✓
2	3	✓
3	3	✓
4	3	✓
5	1	✓
6	1	✓
7	1	✓
8	1	✓
9	1	✓

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 1.5 ✖ [2] .

0	4	✓
1	4	✓
2	3	✓
3	3	✓
4	3	✓
5	1	✓
6	1	✓
7	1	✓
8	1	✓
9	1	✓
10	1	✓
11	2	✓
12	2	✓
13	2	✓
14	2	✓

Se execută programul de mai jos:

```
sem_t globalSem;

void *thFun(void *arg)
{
    long id = (long)arg;
    sleep(id);
    sem_wait(&globalSem);
    sleep(SLEEP_TIME);
    sem_post(&globalSem);
    return NULL;
}

int main()
{
    long i;
    pthread_t tid[NR_THREADS];
    sem_init(&globalSem, 0, SEM_VALUE);
    for(i=0; i<NR_THREADS; i++) {
        pthread_create(&tid[i], NULL, thFun, (void*)i);
    }
    for(i=0; i<NR_THREADS; i++) {
        pthread_join(tid[i], NULL);
    }
    sem_destroy(&globalSem);
    return 0;
}
```

Știind că valorile pentru constante sunt NR_THREADS=7, SEM_VALUE=2 și SLEEP_TIME=5, completați durata de execuție (în secunde) a fiecărui thread:

0	5	✓
1	6	✓
2	12	✗ [10]
3	13	✗ [11]
4	14	✗ [15]
5	15	✗ [16]
6	16	✗ [20]

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 1KB. Spațiul virtual de adrese al fiecărui proces este de 256KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 128KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	6
3	5
4	9
8	7

Numărul de pagini din memoria virtuală a fiecărui proces este: ✖ [256] .

Numărul de cadre din memoria fizică a sistemului este: ✖ [128] .

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✖ [18] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
5045	-1 ✖ [10165]
2220	-1 ✓
4948	-1 ✖ [10068]
3236	-1 ✖ [5284]

Se dă un planificator de tipul **First Come, First Served**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	3	3
2	0	6
3	2	2
4	4	4

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: ✓ .

0	2	✓
1	2	✓
2	2	✓
3	2	✓
4	2	✓
5	2	✓
6	3	✓
7	3	✓
8	1	✓
9	1	✓
10	1	✓
11	4	✓

?

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: ✓ .

0	2	✓
1	2	✓
2	2	✓
3	2	✓
4	2	✓
5	2	✓
6	3	✓
7	3	✓
8	1	✓
9	1	✓
10	1	✓
11	4	✓
12	4	✓
13	4	✓
14	4	✓

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 4KB. Spațiul virtual de adrese al fiecărui proces este de 64KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 512KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	4
3	5
6	2
7	9

Numărul de pagini din memoria virtuală a fiecărui proces este: ✖ [16] .

Numărul de cadre din memoria fizică a sistemului este: ✖ [128] .

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✖ [16] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
36522	-1 ✓
29127	-1 ✖ [37319]
31962	-1 ✖ [40154]
29811	-1 ✖ [38003]

În urma execuției codului de mai jos, ia naștere o ierarhie de procese, reprezentată ca un arbore în care procesul inițial este rădăcina.

```
if(fork() == 0) {  
    fork();  
} else {  
    fork();  
    fork();  
}
```

Numărul de procese din arbore este ✖ [6] .

Adâncimea arborelui (ca număr de noduri de la rădăcină până la cea mai îndepărtată frunză) este ✖ [3] .

Numărul de frunze din arbore este ✔ .

Numărul maxim de copii ai unui nod este ✖ [3] .

Ce va afișa codul de mai jos?

```
#define SIZE 7

typedef struct {
    int id;
    sem_t *sem1;
    sem_t *sem2;
} TH_PARAMS;

void *th_fun(void *arg)
{
    TH_PARAMS *p = (TH_PARAMS*)arg;
    sem_wait(p->sem1);
    printf("%d\n", p->id);
    sem_post(p->sem2);
    return NULL;
}

int main()
{
    sem_t sem[SIZE];
    TH_PARAMS p[SIZE];
    pthread_t tid[SIZE];
    int v1[SIZE] = {1, 5, 6, 2, 4, 3, 0};
    int v2[SIZE] = {5, 4, 2, 3, 1, 0, 6};
    int i;

    for(i=0; i<SIZE; i++) {
        sem_init(&sem[i], 0, 0);
    }
    for(i=0; i<SIZE; i++) {
        p[i].id = i;
        p[i].sem1 = &sem[v1[i]];
        p[i].sem2 = &sem[v2[i]];
        pthread_create(&tid[i], NULL, th_fun, &p[i]);
    }
    sem_post(&sem[2]);
    for(i=0; i<SIZE; i++) {
        pthread_join(tid[i], NULL);
    }
    return 0;
}
```

2 ✖ [3]

3 ✖ [5]

5 ✖ [6]

6 ✖ [2]

(hint: dacă crezi că e nevoie de mai multe căsuțe, mai urmărește odată codul)

Ne aflăm în directorul curent **/home/guest**. Pentru fiecare din căile de mai jos, completați varianta normalizată a acesteia, adică scrieți calea absolută spre fișierul indicat, în cel mai simplu mod posibil.

- `../code/exercise/exam/source/Homework/../../../../lab/./question/laborator/Homework/./2023/repos/../../../../`
 ✖
`[/home/code/exercise/question/laborator/Homework/exam/ex2.txt]`
- `laborator/./problems/2023/code/question/so/../../../../code/ex1.txt`
 ✖
`[/home/guest/laborator/problems/2023/code/ex1.txt]`
- `lab/./pages/./question/exam/repos/problems/../../../../older/laborator/laborator/./source/2023/./code`
 ✖
`[/home/guest/lab/question/older/laborator/source/2023/code.c]`
- `/media/pages/./question/question/exercise/Homework/../../../../exam/Documents/./2023/./Downloads/ex2..`
 ✖
`[/media/question/exam/Documents/Downloads/ex2.txt]`

```
unsigned char c = 0;
int i = 0, pos = 0;
unsigned int x = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if(++i % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
pos = lseek(fd, 0, SEEK_CUR);
printf("pos = %d\n", pos);
lseek(fd, -4, SEEK_CUR);
read(fd, &x, 4);
printf("x = %d\n", x);
lseek(fd, 0, SEEK_SET);
read(fd, &c, 1);
printf("c1 = %02x\n", c);
while(read(fd, &c, 1) == 1) {
    if(c == 0) break;
}
read(fd, &c, 1);
printf("c2 = %02x\n", c);
```

Completați rezultatul afișat pe ecran.

```
4b 50 4b 58 44 4c 56 53 4b 58 48 51 52 48 41 4d
53 5a 59 57 48 49 56 59 44 4d 4c 51 51 00 32 4a
56 53 44 43 54 43 43 4b 57 58 47 48 46 41 59 46
52 45 50 26 01 00 00
pos =  ✓
x =  ✖ [294]
c1 =  ✓
```



Completați rezultatul afișat pe ecran.

4b 50 4b 58 44 4c 56 53 4b 58 48 51 52 48 41 4d
53 5a 59 57 48 49 56 59 44 4d 4c 51 51 00 32 4a
56 53 44 43 54 43 43 4b 57 58 47 48 46 41 59 46
52 45 50 26 01 00 00

pos = ✓

x = ✗ [294]

c1 = ✓

c2 = ✓